Graficación de primitivas 2D
(1 / 2)

Computación gráfica y ambientes virtuales (0494)

Facultad de Ingeniería
Centro Regional Universitario Córdoba - IUA
Universidad de la Defensa Nacional

2026





Generación de imágenes

Recordando los conceptos de Computación gráfica

- La **computación gráfica** es la ciencia (y el arte) de comunicar visualmente por medio de una pantalla y los dispositivos de interacción de una computadora
- Es la **creación**, **manipulación** y **almacenamiento** de modelos de objetos e imágenes (Fetter)

Debemos estudiar:

- Como modelar las formas
- Como modelar la apariencia de las formas
- Como visualizar las formas

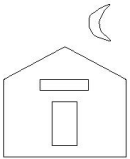
Concepto de "rendering"

- Es el proceso que permite obtener una representación estática 2d (imagen) de un mundo abstracto 3d

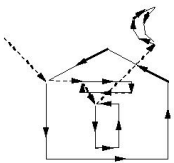
Repaso técnicas de visualización

Vector o caligráfica

- Se dibujan segmentos de lineas para representar las formas
- Velocidad de pintado depende de la complejidad de la imagen
- Muy costoso para rellenar formas (hay que generar nuevos segmentos de línea)



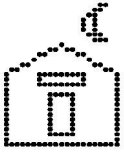
Ideal Drawing



Vector Drawing

Raster (trama)

- Se dibujan puntos que unidos representan las formas
- Velocidad de pintado independiente de la complejidad de la imagen
- Es posible rellenar formas de manera sencilla y sin costo de pintado adicional



outline primitives

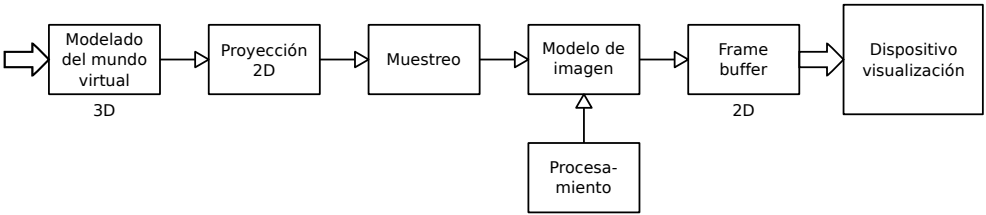


filled primitives

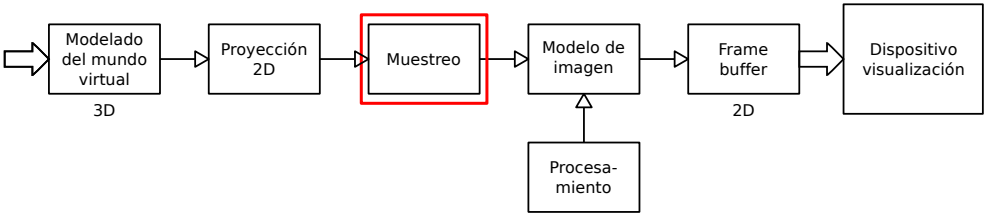
Rendering pipeline

Conjunto de procesos que permiten visualizar un modelo 3d abstracto por medio de una representación 2d.

- Transformación del modelo matemático del mundo a pixeles en la pantalla
- Este proceso requiere múltiples pasos que se realizan en secuencia
- La información salida de una etapa sirve como información de entrada de la siguiente



Digitalización de primitivas 2D



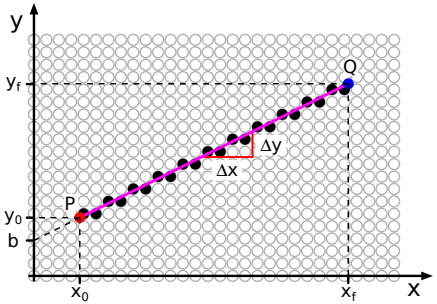
Elemento básico para representar en 2d las formas: líneas

- Segmentos de recta
- Cónicas
  - Círculos y elipses
  - Parábolas
- Polinomios y splines

Segmentos de recta

Problema

"Dados dos puntos **P** y **Q** en el plano se desea dibujar la línea que une ambos puntos en un sistema raster."



- La línea tiene una descripción matemática continua

Ec. de la recta: forma algebraica

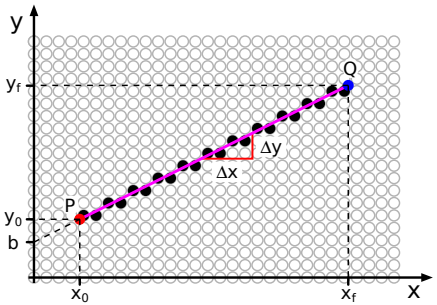
$$y(x) = m \cdot x + b$$

$$m = \frac{y_f - y_0}{x_f - x_0} \equiv \frac{\Delta y}{\Delta x}$$

Segmentos de recta

Problema

"Dados dos puntos **P** y **Q** en el plano se desea dibujar la línea que une ambos puntos en un sistema raster."



- Hay que decidir cuales pixeles deben pintarse en pantalla
- Se realiza un proceso de transformación para pasar de un dominio continuo a un dominio discreto (muestreo)
- Hay que escribir los pixeles resultantes siguiendo la líneas del sistema raster
- Por esto se denomina "conversión por barrido de línea" (*scan-conversion line*)

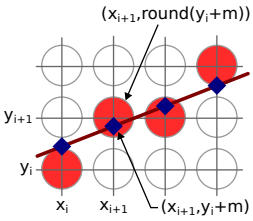
Segmentos de recta

Algoritmos para segmentos de recta

- Algoritmo incremental o DDA (Digital Differential Analyzer)
- Algoritmo de punto medio o de línea de Bresenham

Algoritmo incremental (DDA)

En este algoritmo la línea se muestrea a intervalos unitarios en una dirección y se calcula el valor entero más cercano en la otra coordenada según la dirección de la línea.



$$\frac{\Delta y}{\Delta x} = m \quad \Rightarrow \quad \Delta y = m \cdot \Delta x$$

$$\text{como} \quad \Delta x = 1 \quad \Rightarrow \quad \Delta y = m$$

$$y_{i+1} = y_i + \Delta y$$

$$y_{i+1} = y_i + m$$

$$\text{pixel coord} = (x_{i+1}, round(y_{i+1}))$$

$$round(x) = floor(x + 0.5)$$

Código de ejemplo

```
inline int round(const float x)
{
    return int(floor(x + 0.5f));
}

void drawLineDDA(int x0, int y0, int x1, int y1)
{
    int dx = x1 - x0;
    int dy = y1 - y0;

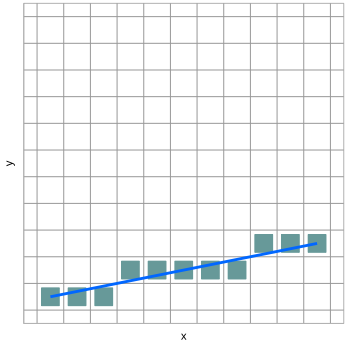
    const float m = (float)dy / dx;

    int x;
    float y = (float)y0;

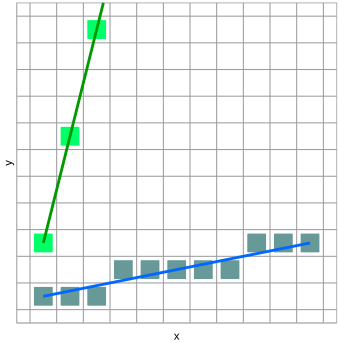
    for (x = x0 ; x < x1 ; x++) {
        write_pixel(x, (int)round(y));
        y += m;
    }
}
```

Algoritmo incremental (DDA)

Caso m $<$ 1



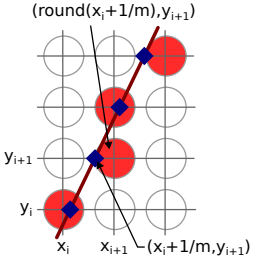
Caso m $>$ 1



Algoritmo incremental (DDA)

Modificación estrategia de avance

Podemos considerar hacer incrementos unitarios en la dirección y y calcular la coordenada en la dirección x.



$$\frac{\Delta y}{\Delta x} = m \quad \Rightarrow \quad \Delta x = \frac{1}{m} \cdot \Delta y$$

$$\text{como} \quad \Delta y = 1 \quad \Rightarrow \quad \Delta x = \frac{1}{m}$$

$$x_{i+1} = x_i + \Delta x$$

$$x_{i+1} = x_i + \frac{1}{m}$$

$$\text{pixel coord} = (round(x_{i+1}), y_{i+1}))$$

Código de ejemplo

```
inline int round(const float x)
{
    return int(floor(x + 0.5f));
}

void drawLineDDA(int x0, int y0, int x1, int y1)
{
    int dx = x1 - x0;
    int dy = y1 - y0;

    const float m = (float)dy / dx;

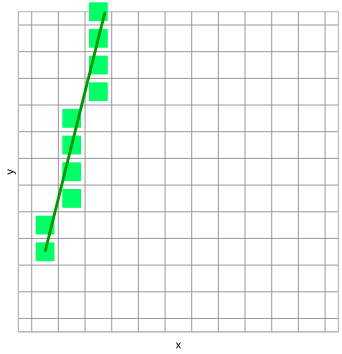
    float x = (float)x0;
    int y;

    for (y = y0 ; y < y1 ; y++) {
        write_pixel(round(x), y);
        x += 1.0f / m;
    }
}
```

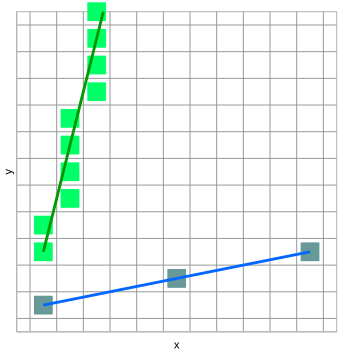
Algoritmo incremental (DDA)

Modificación estrategia de avance

Caso m $>$ 1



Caso m $<$ 1



Algoritmo incremental (DDA)

Versión final del algoritmo

```
inline int round(const float x)
{
    return int(floor(x + 0.5f));
}

void drawLineDDA(int x0, int y0, int x1, int y1)
{
    int dx = x1 - x0;
    int dy = y1 - y0;
    int steps = fabs(dx) > fabs(dy) ? fabs(dx) :
          fabs(dy);

    float xIncrement = dx / float(steps);
    float yIncrement = dy / float(steps);

    float x = x0;
    float y = y0;

    for (int i = 0; i <= steps; i++) {
        write_pixel(round(x), round(y));
        x += xIncrement;
        y += yIncrement;
    }
}
```

Ventajas y desventajas

Ventajas

- Implementación más rápida que si se implementa la ec. $y = m \cdot x + b$

Desventajas

- Requiere utilización de operaciones de punto flotante que pueden ser costosas
- Se acumula error de redondeo que puede hacer diverger las posiciones de los pixeles para líneas largas

Algoritmo de Bresenham

- Desarrollado en 1965 por Jack Bresenham en IBM
- Algoritmo para muestrear una línea de manera más eficiente usando unicamente aritmética de números enteros.
- Adecuado para ser implementado en hardware gráfico de bajo nivel
- Este algoritmo (y algunas variantes) es el que se encuentra implementado los procesadores gráficos actuales

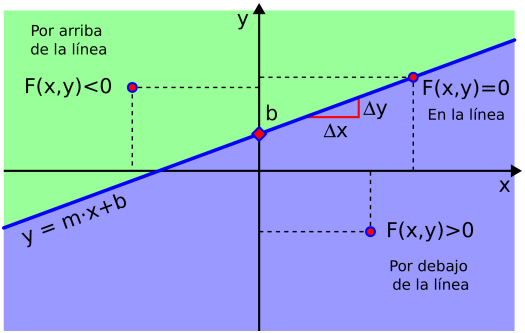
Ec. de la recta: forma implícita

$$y(x) = m \cdot x + b = \frac{\Delta y}{\Delta x} \cdot x + b$$

$$F(x, y) = \Delta y \cdot x - \Delta x \cdot y + \Delta x \cdot b = 0$$

- $y(x)$ y $F(x, y) = 0$ representan la misma línea
- Cualquier punto (x, y) que satisfaga $F(x, y) = 0$ pertence a la línea
- F no nos dice como calcular y dado x (o al revés)
- El signo de $F(x, y)$ define en qué región del plano se encuentra el punto

Algoritmo de Bresenham

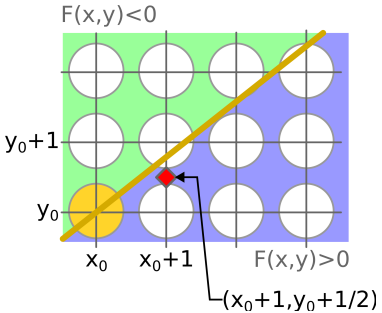


El signo de $F(x, y)$ se puede usar para identificar tres regiones en el plano:

- $F(x, y) < 0$: región por encima de la línea
- $F(x, y) = 0$: puntos sobre la línea
- $F(x, y) > 0$: región por debajo de la línea

El algoritmo de Bresenham aprovecha esta propiedad para determinar qué pixel aproxima mejor una línea usando operaciones de enteros unicamente.

Algoritmo de Bresenham



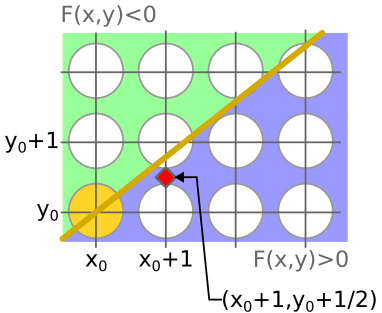
Hipótesis

- La pendiente m es positiva menor que uno $(0 < m < 1)$
- La recta crece hacia la "derecha" $(x_0 < x_{0+1}$ y $y_0 < y_{0+1})$
- La línea pasa por el punto (x_0, y_0)

Planteo del algoritmo

- Hay que decidir cuál es el próximo pixel a pintar
- Se toma el punto medio entre los dos posibles pixeles y se evalúa el signo de $F(x, y)$
- El valor que tome $F(x, y)$ lo llamaremos *variable de decisión* "D"
- La magnitud no importa ya que nos interesa el signo

Algoritmo de Bresenham



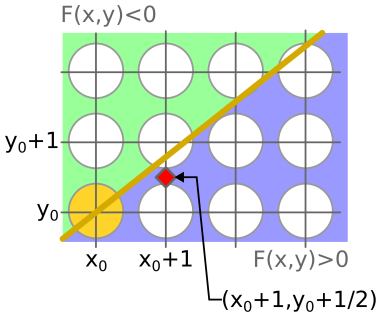
ec. implícita $\rightarrow F(x,y) = \Delta y \cdot x - \Delta x \cdot y + \Delta x \cdot b$

$$
\begin{aligned}
F(x_0+1, y_0+\frac{1}{2}) &= \Delta y \cdot (x_0+1) - \Delta x \cdot (y_0+\frac{1}{2}) + \Delta x \cdot b \\
&= (\Delta y \cdot x_0 - \Delta x \cdot y_0 + \Delta x \cdot b) + (\Delta y - \frac{\Delta x}{2}) \\
&= F(x_0, y_0) + (\Delta y - \frac{\Delta x}{2}) \\
&= 0 + \Delta y - \frac{\Delta x}{2}
\end{aligned}
$$

Multiplicando por 2 ambos miembros para evitar la división se obtiene la *variable de decisión* D

$$\Rightarrow D = 2 \cdot \Delta y - \Delta x$$

Algoritmo de Bresenham



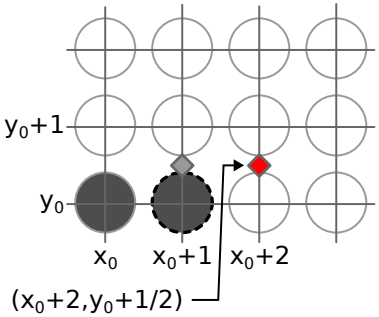
Se analiza el signo de D:

- Si $D > 0$ el punto medio está por debajo de la línea: debo pintar el pixel $(x_0 + 1, y_0 + 1)$ (adyacente superior)
- Si $D < 0$ el punto medio está por encima de la línea: debo pintar el pixel $(x_0 + 1, y_0)$ (adyacente)

¿Es posible actualizar la variable de decisión mediante operaciones aritméticas de enteros?

- Se analiza el valor de la variable de decisión D para la columna $x_0 + 2$
- Se plantean las dos posibilidades dependiendo qué pixel (filas) se seleccione en el paso actual.

Algoritmo de Bresenham



Pixel adyacente

- El próximo punto medio para evaluar $F(x, y)$ se encontrará en $(x_0 + 2, y_0 + 1/2)$
- Se plantea cómo actualizar el valor de D para el siguiente paso

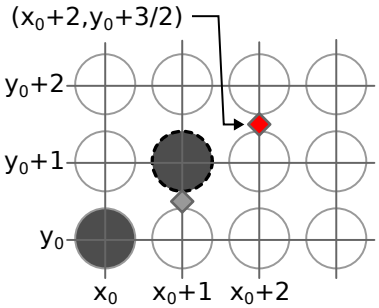
$$D_{x_0+2} = D_{x_0+1} + \Delta D$$
$$\Delta D = D_{x_0+2} - D_{x_0+1}$$

$$F(x_0 + 2, y_0 + \frac{1}{2}) - F(x_0 + 1, y_0 + \frac{1}{2}) = \Delta y$$

Multiplicamos por 2 para poder sumarlo a la definición previa de la variable de decisión.

$$\Rightarrow \Delta D_{ady} = 2 \cdot \Delta y$$

Algoritmo de Bresenham



Pixel adyacente superior

- El próximo punto medio para evaluar $F(x, y)$ se encontrará en $(x_0 + 2, y_0 + 3/2)$
- Se plantea cómo actualizar el valor de D para el siguiente paso

$$\Delta D = D_{x_0+2} - D_{x_0+1}$$

$$F(x_0 + 2, y_0 + \frac{3}{2}) - F(x_0 + 1, y_0 + \frac{1}{2}) = \Delta y - \Delta x$$

Multiplicamos por 2 para poder sumarlo a la definición previa de la variable de decisión.

$$\Rightarrow \Delta D_{aSup} = 2 \cdot (\Delta y - \Delta x)$$

Algoritmo de Bresenham

Resumen del algoritmo

- Válido para números enteros y pendiente $0 < m < 1$ y $x_0 < x_1 \;\; y_0 < y_1$!
- Analizar el signo de la *variable de decisión* para pintar el siguiente pixel
- Actualizar la *variable de decisión* según el pixel que se pinta (adyacente ó adyacente superior)

```
void drawLineBresenham(int x0, int y0, int x1, int y1){
    int dx = x1 - x0;
    int dy = y1 - y0;

    int des_var = 2 * dy - dx;
    const int dd_ady = 2 * dy;
    const int dd_asup = 2 * (dy - dx);

    const int steps = dx;

    int x = x0;
    int y = y0;

    write_pixel(x,y);

    for (int i = 0; i < steps; i++) {
        x++;
        if( des_var < 0 ){
            des_var += dd_ady;
        } else {
            des_var += dd_asup;
            y++;
        }
        write_pixel(x, y);
    }
}
```

Algoritmo de Bresenham

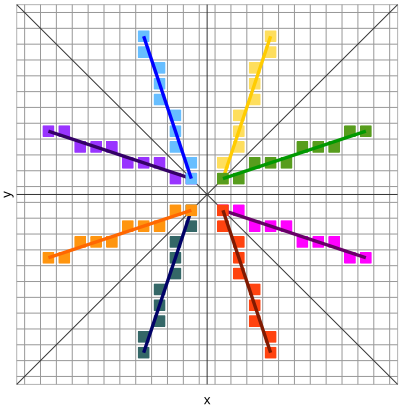
Extensión para todos los casos

Pendiente mayores que 1 $(m > 1)$

- Se usa misma estrategia que algoritmo DDA: se avanza en dirección "y" y se calcula "x"

Pendientes negativas

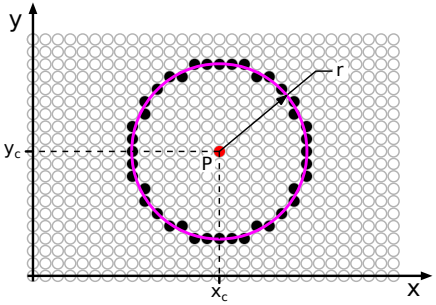
- Se utiliza el concepto de simetría y se hace disminuir la posición en "x" ó "y" del pixel según el signo del Δx ó del Δy respectivamente
- Esta modificación permite también alterar el sentido de trazado de la línea ($x_0 > x_1$ y/o $y_0 > y_1$)



Circunferencias

Problema

"Dado un punto **P** en el plano se desea dibujar una linea curva cuyos puntos se encuentran a una distancia r respecto de **P** (circunferencia) en un sistema raster."



- Una circunferencia tiene una descripción matemática continua

Ec. de una circunferencia

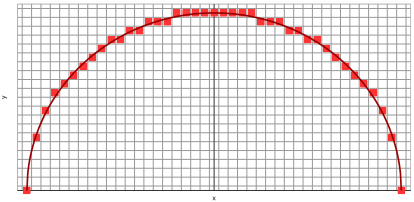
$$r^2 = (x - x_c)^2 + (y - y_c)^2$$

Circunferencias

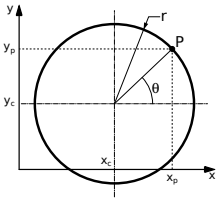
Discretización usando incrementos unitarios en x

$$y(x) = y_c \pm \sqrt{r^2 - (x - x_c)^2}$$

$$y_{k+1} = y_c \pm \sqrt{r^2 - (x_k + \Delta x - x_c)^2}$$



Discretización usando coordenadas polares



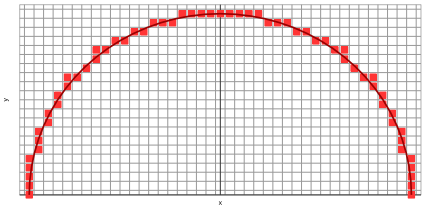
$$x = x_c + r \cdot \cos(\theta)$$

$$y = y_c + r \cdot \sin(\theta)$$

$$\theta_{k+1} = \theta_k + \Delta\theta$$

$$x_{k+1} = x_c + r \cdot \cos(\theta_{k+1})$$

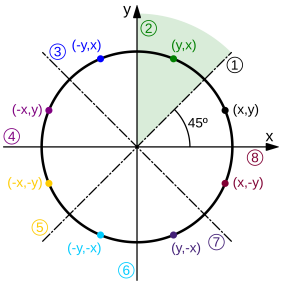
$$y_{k+1} = y_c + r \cdot \sin(\theta_{k+1})$$



Circunferencias

Reducción costo computacional mediante simetría

- Se puede aprovechar la simetría de la circunferencia para realizar el proceso de digitalización
- Una circunferencia presenta simetría por cuadrantes y por *octantes*
- Aprovechando esto se puede calcular los pixels entre $x = 0$ y $x = y$ (2do octante!) y luego construir el resto de la circunferencia



Conclusiones algoritmos directos

- Costosos computacionalmente (punto flotante, raíz cuadrada, op. trigonométricas)
- Espaciado no uniforme de los puntos
- ¿Es posible utilizar un algoritmo incremental con operaciones simples sobre enteros?

Algoritmo de punto medio para circunferencias

- Mismo planteo que el utilizada para trazar segmentos rectos:
  - Muestrear usando avances unitarios
  - Determinar pixel más cercano a la circunferencia

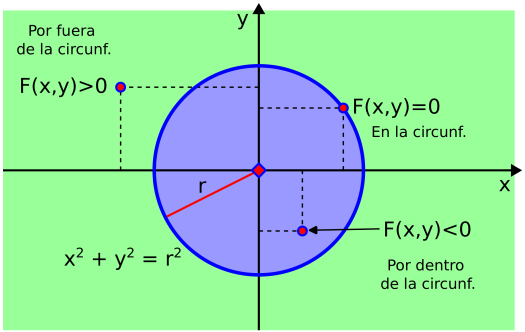
Hipótesis

- Valores enteros para la posición del centro (x_c, y_c) y el radio r de la circunferencia

Estrategia

- Se calculan las posiciones de los pixeles para una circunferencia de radio r centrada en el origen $(0, 0)$
- Luego se desplaza posición sumando las coordenadas del centro de la circunferencia (x_c, y_c)

Algoritmo de punto medio para circunferencias



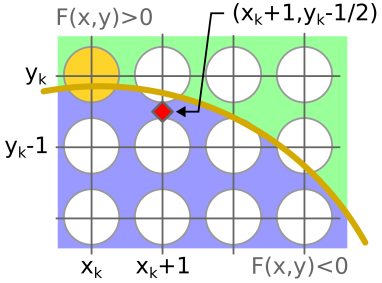
Ec. implícita de la circunferencia

$$F(x, y) = x^2 + y^2 - r^2$$

El signo de $F(x, y)$ se puede usar para identificar tres regiones en el plano:

- $F(x, y) < 0$: región adentro de la circunf.
- $F(x, y) = 0$: puntos sobre la circunf.
- $F(x, y) > 0$: región fuera de la circunf.

Algoritmo de punto medio para circunferencias



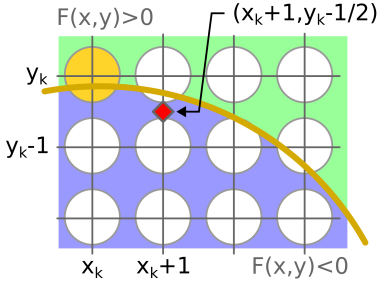
Hipótesis

- Se calculan los pixels para el octante 2 ($0 \leq x \leq y$)
- La pendiente m varía entre 0 y -1 ($0 \leq m \leq -1$)
- Se pintó el pixel correspondiente a la posición (x_k, y_k)

Planteo del algoritmo

- Hay que decidir cuál es el próximo pixel a pintar
- Se toma el punto medio entre los dos posibles pixeles y se evalúa el signo de $F(x, y)$
- El valor que tome $F(x, y)$ lo llamaremos *variable de decisión* "D"
- La magnitud no importa ya que nos interesa el signo

Algoritmo de punto medio para circunferencias



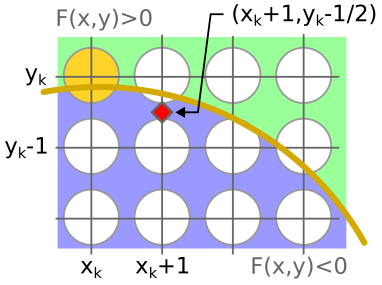
ec. implícita $\rightarrow F(x,y) = x^2 + y^2 - r^2$

$$F(x_k + 1, y_k - \frac{1}{2}) = (x_k + 1)^2 + (y_k - \frac{1}{2})^2 - r^2$$
$$= (x_k^2 + y_k^2 - r^2) + (2 \cdot x_k - y_k + \frac{5}{4})$$
$$= F(x_k, y_k) + (2 \cdot x_k - y_k + \frac{5}{4})$$

Multiplicando por 4 ambos miembros para evitar la división se obtiene la *variable de decisión* D

$$\Rightarrow D_k = 4 \cdot F(x_k, y_k) + (8 \cdot x_k - 4 \cdot y_k + 5)$$

Algoritmo de punto medio para circunferencias



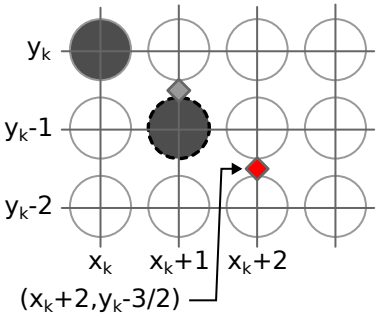
Se analiza el signo de D:

- Si $D > 0$ el punto medio está fuera de la circunf.: debo pintar el pixel $(x_k + 1, y_k - 1)$ (adyacente inferior)
- Si $D < 0$ el punto medio está dentro de la circunf.: debo pintar el pixel $(x_k + 1, y_k)$ (adyacente)

¿Es posible actualizar la variable de decisión mediante operaciones aritméticas de enteros?

- Se analiza el valor de la variable de decisión D para la columna $x_k + 2$
- Se plantean las dos posibilidades dependiendo qué pixel (filas) se seleccione en el paso actual.

Algoritmo de punto medio para circunferencias



Pixel adyacente inferior

- El próximo punto medio para evaluar $F(x, y)$ se encontrará en $(x_k + 2, y_k - 3/2)$
- Se plantea cómo actualizar el valor de D para el siguiente paso

$$F(x_k + 2, y_k - \frac{3}{2}) = (x_k + 2)^2 + (y_k - \frac{3}{2})^2 - r^2$$

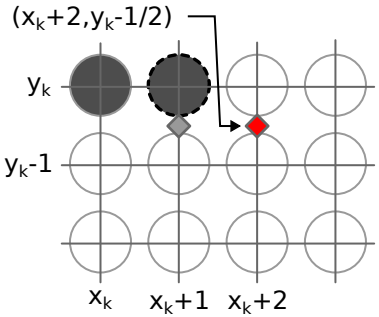
$$F(x_k + 2, y_k - \frac{3}{2}) = D_k + (2 \cdot (x_k + 1) - 2 \cdot y_k + 3)$$

$$\Rightarrow (\Delta D_k)_{aInf} = 2 \cdot (x_k + 1) - 2 \cdot y_k + 3$$

Multiplicamos por 4 para poder sumarlo a la definición previa de la variable de decisión.

$$\Rightarrow (\Delta D_k)_{aInf} = 8 \cdot (x_k + 1) - 8 \cdot y_k + 12$$

Algoritmo de punto medio para circunferencias



Pixel adyacente

- El próximo punto medio para evaluar $F(x, y)$ se encontrará en $(x_k + 2, y_k - 1/2)$
- Se plantea cómo actualizar el valor de D para el siguiente paso

$$F(x_k + 2, y_k - \frac{1}{2}) = (x_k + 2)^2 + (y_k - \frac{1}{2})^2 - r^2$$

$$F(x_k + 2, y_k - \frac{1}{2}) = D_k + (2 \cdot (x_k + 1) + 1)$$

$$\Rightarrow (\Delta D_k)_{ady} = 2 \cdot (x_k + 1) + 1$$

Multiplicamos por 4 para poder sumarlo a la definición previa de la variable de decisión.

$$\Rightarrow (\Delta D_k)_{ady} = 8 \cdot (x_k + 1) + 4$$

Algoritmo de punto medio para circunferencias

Valores iniciales para el algoritmo

- Circunferencia centrada en el origen y valor entero para el radio r
- Se evalúa la variable de decisión para el punto $(x_0, y_0) = (0, r)$ (2do octante!)

$$\begin{aligned} D_0 &= 4 \cdot F(x_0, y_0) + (8 \cdot x_0 - 4 \cdot y_0 + 5) \\ &= 4 \cdot 0 + (8 \cdot 0 - 4 \cdot r + 5) \end{aligned}$$

$$\Rightarrow D_0 = 5 - 4 \cdot r$$

- Se evalúan los incrementos en la variable de decisión

$$\begin{aligned} (\Delta D_0)_{aInf} &= 8 \cdot (x_0 + 1) - 8 \cdot y_0 + 12 \\ &= 8 \cdot (0 + 1) - 8 \cdot r + 12 \end{aligned}$$

$$\Rightarrow (\Delta D_0)_{aInf} = 20 - 8 \cdot r$$

$$\begin{aligned} (\Delta D_0)_{ady} &= 8 \cdot (x_0 + 1) + 4 \\ &= 8 \cdot (0 + 1) + 4 \end{aligned}$$

$$\Rightarrow (\Delta D_0)_{ady} = 12$$

Algoritmo de punto medio para circunferencias

Resumen del algoritmo

- Circunferencia centrada en el origen!
- Válido para números enteros (radio de la circunferencia)
- Sólo calcula los pixeles para 1/8 de la circunferencia (2do octante)!
- Analizar el signo de la *variable de decisión* para pintar el siguiente pixel
- Actualizar los incrementos para la *variable de decisión*

```
void drawCircleMP(int r){
    int des_var = 5 - 4 * r;
    int dd_ady = 12;
    int dd_ainf = 20 - 8 * r;

    int x = 0;
    int y = r;

    write_pixel(x,y);
    x++;

    while( y > x ){
        if( des_var < 0 ){
            des_var += dd_ady;
        } else {
            des_var += dd_ainf;
            y--;
        }
        // Dk = 4 * (2 (x + 1) + 1)
        dd_ady = 8 * x + 12;
        // D'k = 4 * (2 * (x + 1) - 2 * y + 3)
        dd_ainf = 8 * x - 8 * y + 20;

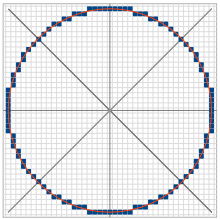
        write_pixel(x, y);
        x++;
    }
}
```

Algoritmo de punto medio para circunferencias

Extensión para todos los casos

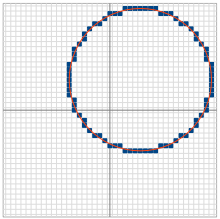
Circunferencia completa

- Se calculan el resto de los pixeles aplicando las condiciones de simetría de octantes



Posición del centro de la circunferencia

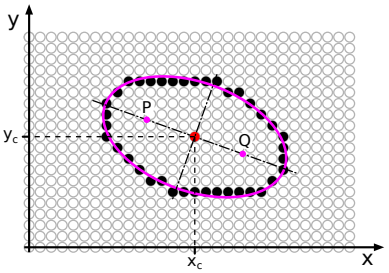
- Se calcula la circunferencia centrada en el origen y luego se realiza una transformación de desplazamiento de los puntos calculados usando las coordenadas del centro de la circunferencia



Elipses

Problema

"Dado dos punto **P** y **Q** en el plano se desea dibujar en un sistema raster una linea curva en donde la suma de las distancias a los punto P y Q para cualquiera de sus puntos es constante (elipse)."



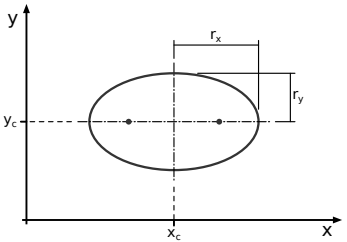
- Una elipse tiene una descripción matemática continua
- P y Q se denominan focos de la elipse
- El eje mayor es el segmento de recta que pasa por los dos focos
- El eje menor es el segmento perpendicular al eje mayor que lo bisecta

Ec. general de una elipse

$$A \cdot x^2 + B \cdot y^2 + C \cdot x \cdot y + D \cdot x + E \cdot y + F = 0$$

Elipses

- Para simplificar el planteo de cómo se digitalizar una elipse se analiza una en la *posición estándar*
- Posición estándar:
  - Eje *mayor* horizontal (paralelo al eje "x")
  - Implica que el eje *menor* es vertical (paralelo al eje "y")



Ec. para elipse en posición estándar

$$\left(\frac{x - x_c}{r_x}\right)^2 + \left(\frac{y - y_c}{r_y}\right)^2 = 1$$

- r_x longitud del semi-eje mayor
- r_y longitud del semi-eje menor

Algoritmo de punto medio para elipses

- Mismos problemas de los algoritmos directos que para líneas y circunferencias (costosos computacionalmente)
- Se busca usar un algoritmo incremental que reduzca la cantidad de operaciones y aproveche la aritmética de enteros
- Se usa misma estrategia que la utilizada para trazar líneas y circunferencias:
  - Muestrear usando avances unitarios
  - Determinar pixel más cercano a la elipse
  - Aprovechar simetría de la figura geométrica

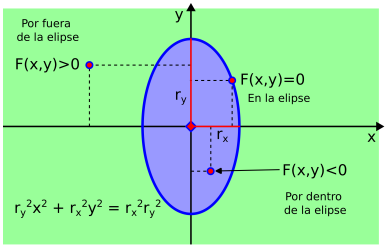
Hipótesis

- Valores enteros para la posición del centro (x_c, y_c) y las dimensiones de los semi-ejes r_x y r_y
- Elipse en posición estándar (ejes de la elipse alineados con los ejes de sistema de referencia)

Algoritmo de punto medio para elipses

Estrategia

- Se calculan las posiciones de los pixeles para una elipse con dimensiones de los semi-ejes r_x y r_y centrada en el origen $(0, 0)$
- Luego se desplaza posición sumando las coordenadas del centro de la elipse (x_c, y_c)
- Luego de la traslación se puede rotar la elipse respecto a su centro para mostrar los ejes alineados a una dirección arbitraria



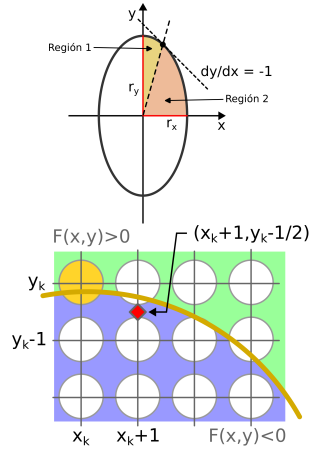
Ec. implícita de la elipse (centrada en origen)

$$F(x, y) = r_y^2 \cdot x^2 + r_x^2 \cdot y^2 - r_x^2 \cdot r_y^2$$

El signo de $F(x, y)$ se puede usar para identificar tres regiones en el plano:

- $F(x, y) < 0$: región adentro de la elipse
- $F(x, y) = 0$: puntos sobre la elipse
- $F(x, y) > 0$: región fuera de la elipse

Algoritmo de punto medio para elipses



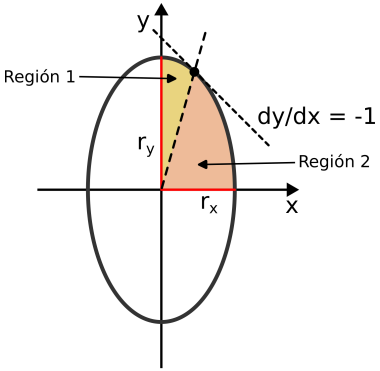
Hipótesis

- Se asume $r_x < r_y$
- Se calculan los pixels para el cuadrante 1 ($0 \leq x \leq r_x$)

Planteo del algoritmo

- Se divide el cuadrante en dos regiones dependiendo del valor de derivada dy/dx
- Hay que decidir cuál es el próximo pixel a pintar
- Se toma el punto medio entre los dos posibles pixeles y se evalúa el signo de $F(x, y)$
- El valor que tome $F(x, y)$ lo llamaremos *variable de decisión* "D"
- La magnitud no importa ya que nos interesa el signo

Algoritmo de punto medio para elipses



Determinación de región del cuadrante

- El límite de las dos regiones se define cuando se alcanza el valor $dy/dx = -1$ para la derivada
- Recorriendo el cuadrante desde $x = 0$ hacia $x = r_x$ el cambio de región 1 a región 2 se da cuando

$$\frac{dy}{dx} = -\frac{2r_y^2 \cdot x}{2r_x^2 \cdot y}$$
$$\Rightarrow 2r_y^2 \cdot x \geq 2r_x^2 \cdot y$$

La región determina como realizar el avance del algoritmo:

- Si estamos en región 1 hay que aumentar en la dirección "x"
- Si estamos en región 2 hay que disminuir en la dirección "y"

Algoritmo de punto medio para elipses

Variables de decisión para región 1

$$F(x_k + 1, y_k - \frac{1}{2}) = r_y^2 \cdot (x_k + 1)^2 + r_x^2 \cdot (y_k - \frac{1}{2})^2 - r_x^2 \cdot r_y^2$$

$$F(x_k + 1, y_k - \frac{1}{2}) = r_y^2 x_k^2 + r_y^2 y_k^2 - r_x^2 r_y^2 + \left[r_y^2 (2x_k + 1) + r_x^2 (\frac{1}{4} - y_k) \right]$$

Multiplicando por 4 ambos miembros para evitar la división, se obtiene la variable de decisión D:

$$\Rightarrow (D_k)_1 = 4 \cdot F(x_k, y_k) + \left[4r_y^2 (2x_k + 1) + r_x^2 (1 - 4y_k) \right]$$

Incremento para las variable de decisión en región 1

$$((\Delta D_k)_{ady})_1 = 4r_y^2 (2x_k + 1) + 8r_y^2$$

$$((\Delta D_k)_{alnf})_1 = 4r_y^2 (2x_k + 1) + 8r_y^2 + 2r_x^2 (1 - 4y_k) + 6r_x^2$$

Nota: ya se encuentran multiplicadas por 4 para eliminar divisiones

Algoritmo de punto medio para elipses

Variables de decisión para región 2

$$F(x_k + \frac{1}{2}, y_k - 1) = r_y^2 \cdot (x_k + \frac{1}{2})^2 + r_x^2 \cdot (y_k - 1)^2 - r_x^2 \cdot r_y^2$$

$$F(x_k + \frac{1}{2}, y_k - 1) = r_y^2 x_k^2 + r_y^2 y_k^2 - r_x^2 r_y^2 + \left[r_y^2 (x_k + \frac{1}{4}) + r_x^2 (-2y_k + 1) \right]$$

Multiplicando por 4 ambos miembros para evitar la división, se obtiene la variable de decisión D:

$$\Rightarrow (D_k)_2 = 4 \cdot F(x_k, y_k) + \left[r_y^2 (4x_k + 1) + 4r_x^2 (1 - 2y_k) \right]$$

Incremento para las variable de decisión en región 2

$$((\Delta D_k)_{ady})_2 = 4r_x^2 (1 - 2y_k) + 8r_x^2$$

$$((\Delta D_k)_{aInf})_2 = 2r_y^2 (4x_k + 1) + 6r_y^2 + 4r_x^2 (1 - 2y_k) + 8r_x^2$$

Nota: ya se encuentran multiplicadas por 4 para eliminar divisiones

Algoritmo de punto medio para elipses

Extensión para todos los casos

Elipses completa

- Se calculan el resto de los pixeles aplicando las condiciones de simetría de cuadrantes

Posición del centro de la elipse

- La elipse se calcula centrada en el origen por lo que se realiza una transformación de desplazamiento de los puntos calculados usando las coordenadas del centro de la elipse

Bibliografía recomendada

- HEARN, D. & BAKER, M. P.. *Computer Graphics with OpenGL*. 4ta ed. Essex (UK): Pearson Education Limited, 2014. cap. 5 Implementation algorithms for graphics primitives and attributes, p. 131-160.
- HUGHES J. F., FOLEY, J. D., VAN DAM, A., FEINER, S. K.. *Computer Graphics, Principles and Practice*. 3ra ed. New Jersey (USA): Addison Wesley Publishing Company, 2014. cap. 1. Introduction, p. 14-15

¿¿¿ PREGUNTAS ???